

Chapter 14

Catch, avoid and shoot

One genre, three games



Today

- a player on one line; spawning on a timer
- **object pools** - and the price of reuse
- waves: tweens, timers, a curved path
- fire rate, invulnerability, explosions, a starfield
- growing a game: levels as data, a boss, power-ups, a HUD scene

The genre

- the player can do almost nothing - the game is in **what comes down the screen**
- **readable** threats: a bomb looks like a bomb
- **rising pressure**: easy, then harder and harder
- **fairness**: small hit boxes, a moment of safety, no point-blank shots

Three versions

One game, three times - each version keeps the last one's ideas and adds more

Fruit Catcher

ch14_catcher_simple

- a player held to the bottom, moved left and right
- a spawn timer (time.addEvent, loop)
- things that fall: physics groups with gravityY
- overlap: catch or be hit
- score, lives, game over
- made with create(), thrown away with destroy()

Space Shooter

ch14_shooter_intermediate

- + the player fires, at a limited fire rate
- + OBJECT POOLS: bullets and enemies reused, not destroyed
- + waves: a row (tweens) and a stream (a repeating timer)
- + harder every wave
- + explosions and particles
- + flashing invulnerability
- + a parallax starfield

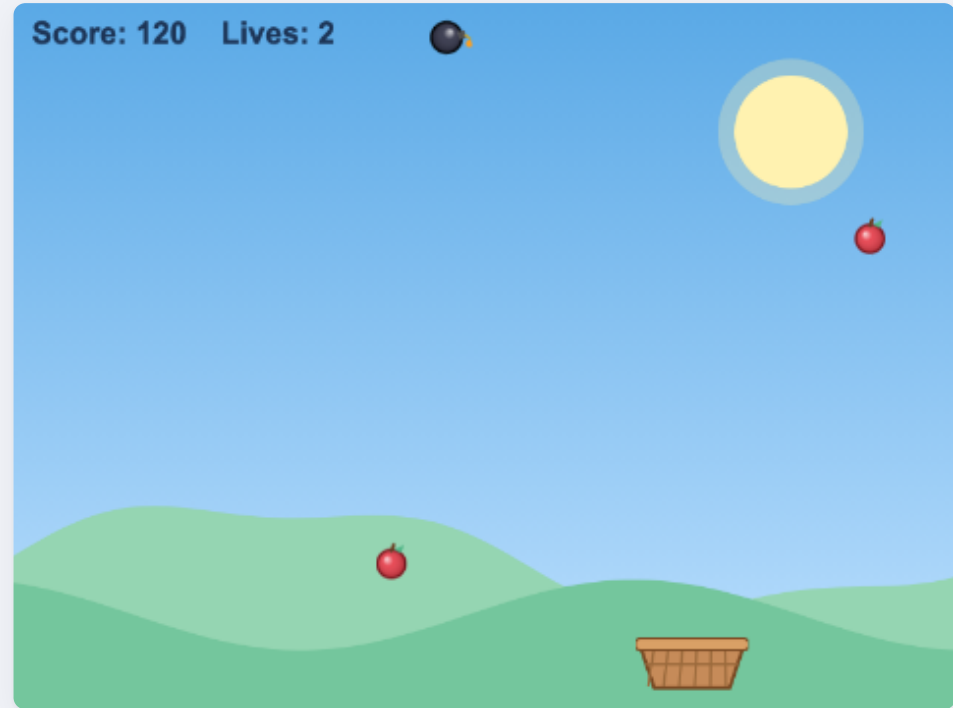
Space Shooter Deluxe

ch14_shooter_advanced

- + levels, described as data
- + a third wave: a curved path
- + enemies that shoot back
- + a boss with a health bar
- + power-ups that wear off
- + a HUD scene fed by events
- + music, preloader, menu
- + high scores that last (localStorage)

Try it: Fruit Catcher

- build and play `ch14_catcher_simple`
- change `SPAWN_DELAY` , `BOMB_CHANCE` , `FALL_GRAVITY` - how does it feel?
- why does a fruit brushing the side not count?



A timer, and things that fall

```
this.fruit = this.physics.add.group({ gravityY: FALL_GRAVITY });

this.spawnTimer = this.time.addEvent({
  delay: SPAWN_DELAY,
  loop: true,
  callback: () => {
    this.dropSomething();
  },
});

this.physics.add.overlap(this.basket, this.fruit, (_basket, fruit) => {
  this.catchFruit(fruit as Phaser.Physics.Arcade.Sprite);
});
```

loop: true - for ever; repeat: n - n + 1 times; remove() - stop

Try it: Space Shooter

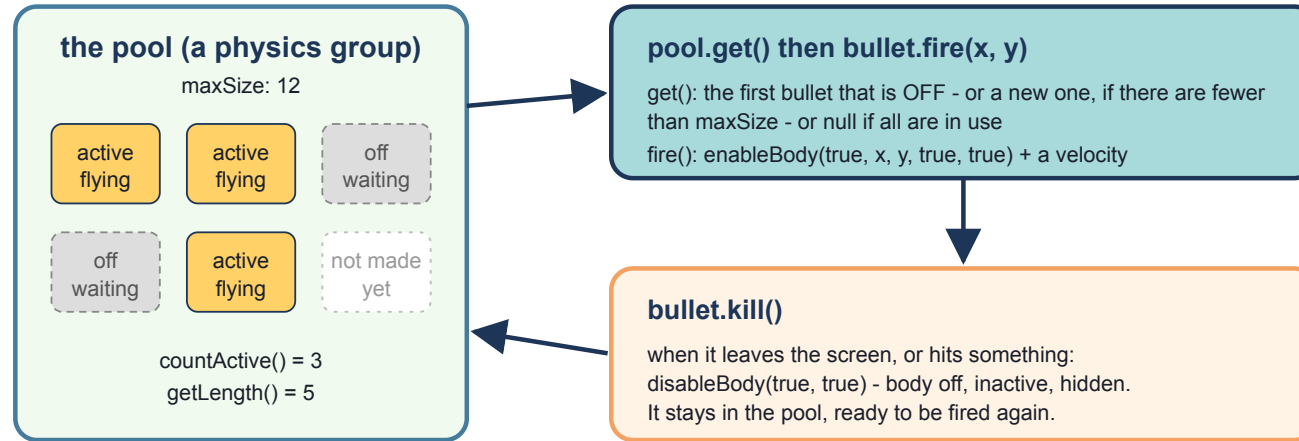
- hold SPACE: about 5 shots a second
- how many bullet **objects** does a minute of firing need?
- (we measured: 9)



Why pool?

- make + destroy hundreds of objects a minute -> the **garbage collector** -> stutter
- instead: make a few, **switch them off**, switch them on again

An object pool: the same few bullets, switched on and off again and again



No new objects while the game runs, and nothing for the garbage collector to clear up - just switches.

A pool in Phaser

```
this.bullets = this.physics.add.group({ classType: Bullet, maxSize: BULLET_POOL_SIZE });

// firing
const bullet = this.bullets.get() as Bullet | null;
if (bullet === null) {
    return;
}
bullet.fire(this.ship.x, this.ship.y - MUZZLE_OFFSET);

// in Bullet
this.enableBody(true, x, y, true, true); // on
this.disableBody(true, true);           // off
```

Three switches

Switch	Off means
<code>active</code>	no <code>preUpdate()</code> ; free to the pool
<code>visible</code>	not drawn
<code>body.enable</code>	not moved, overlaps nothing

`killAndHide` = the first two only. **Demo:** an invisible bullet that still shoots down ships.

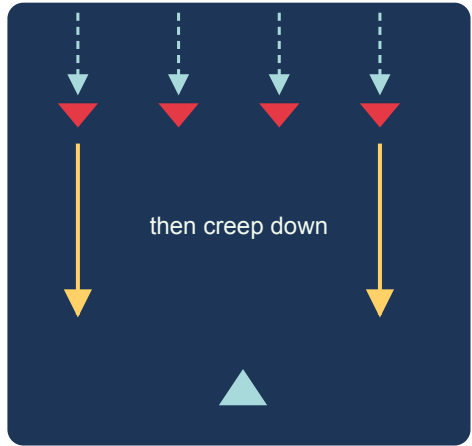
The price of reuse

```
public kill(): void {  
    this.scene tweens.killTweensOf(this);  
    this.disableBody(true, true);  
}
```

- a reused object brings back **everything** you do not reset
- tweens, velocities, angles, tints, fields
- "only goes wrong the second time" = this

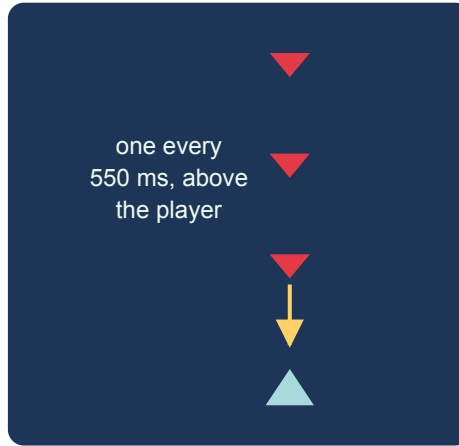
Waves

Three wave patterns



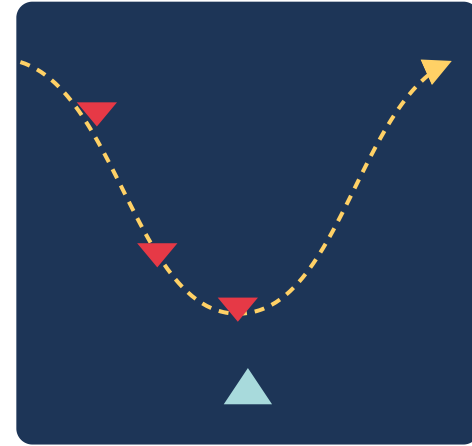
row

tweens, each with a longer delay



stream

a timer with repeat: size - 1



swoop (advanced)

a Spline path; a tween moves 0 to 1 along it

- row: tweens with `delay: i * ROW_STAGGER`
- stream: `addEvent({ delay, repeat: size - 1, ... })`
- count the wave down: `enemiesLeft`, one `enemyGone()`

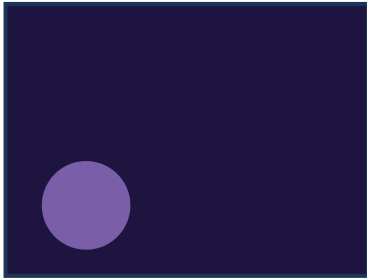
Hit, flash, safe

```
this.scene.tweens.add({
  targets: this,
  alpha: 0.15,
  duration: FLASH_TIME,
  yoyo: true,
  repeat: INVULNERABLE_TIME / (FLASH_TIME * 2) - 1,
  onComplete: () => {
    this.setAlpha(1);
    this.invulnerable = false;
  },
});
```

- a **process callback**: `() => !this.ship.isInvulnerable()`

A parallax starfield

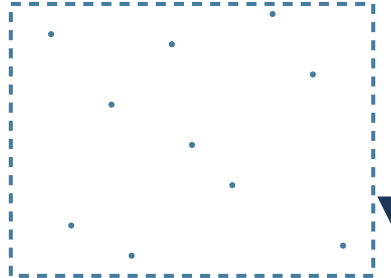
A starfield in three layers - a picture and two TileSprites



space.png

an Image: does not move

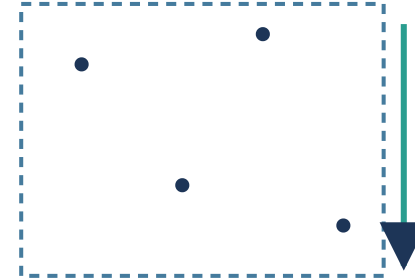
+



far stars

small, dim: 30 pixels per second

+



near stars

bigger, brighter: 90 pixels per second

Each TileSprite repeats a 256 x 256 star picture; changing `tilePositionY` slides the picture inside it.

`TileSprite` + `tilePositionY`; textures drawn with `Graphics` + `generateTexture`

Try it: Space Shooter Deluxe

- three levels, a boss each
- enemies shoot back; power-ups
- HUD scene, music, high scores
- open `levels.ts` - make level 1 easier

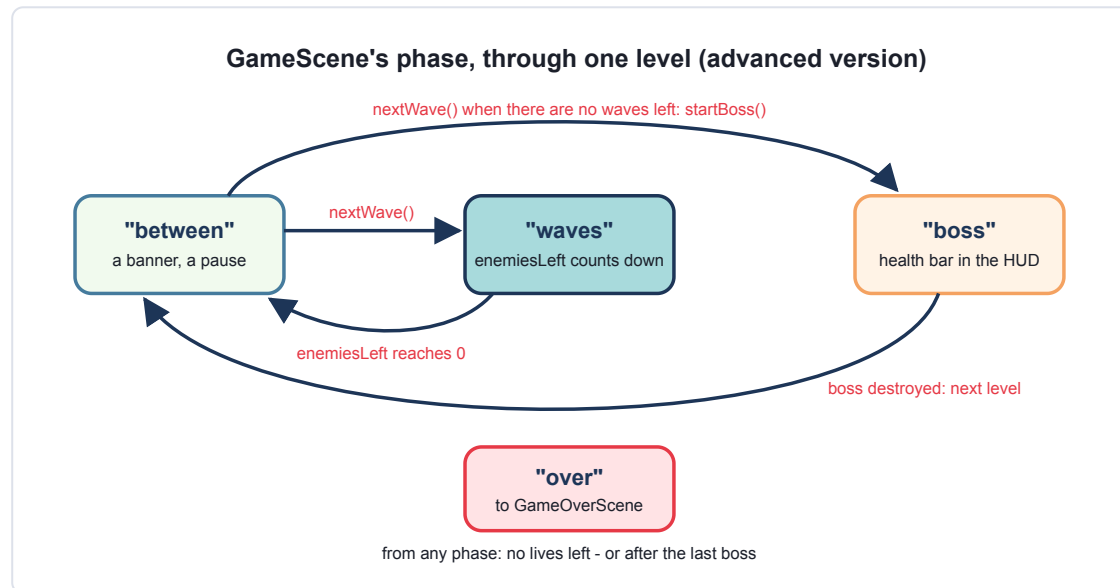


Levels as data, and a phase

```
export type WavePattern = "row" | "stream" | "swoop";

export interface WaveData {
  pattern: WavePattern;
  size: number;           // how many ships
  speed: number;          // pixels per second (swoop: ms)
}

type Phase = "waves" | "between" | "boss" | "over";
```



Along a curve

```
// WaveSpawner.swoop()
const path = new Phaser.Curves.Spline(points);
enemy?.followPath(path, duration, i * SWOOP_GAP);

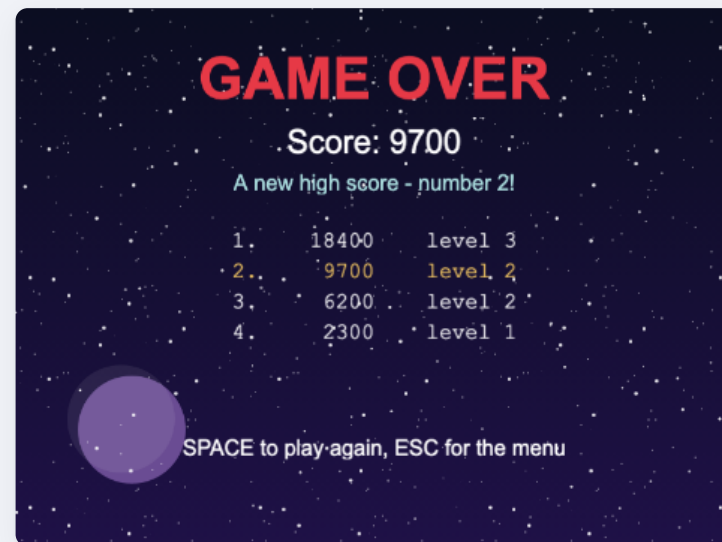
// Enemy.followPath(): a tween moves pathProgress from 0 to 1
this.scene tweens.add({
  targets: this,
  pathProgress: 1,
  duration: duration,
  delay: delay,
});

// Enemy.preUpdate(): every frame, go to that point on the curve
this.path.getPointAt(this.pathProgress, this.point);
this.setPosition(this.point.x, this.point.y);
```

A tween can move **any** number.

Boss, power-ups, HUD

- boss: an entry tween, then `yoyo: true, repeat: -1`; a hit flash with `setTint(0xffffffff).setTintMode(Phaser.TintModes.FILL)` (no `setTintFill` in Phaser 4)
- power-ups: "active until" **times**; `Record<PowerUpKind, string>`; narrowing in `collect()`
- HUD: `this.events.emit(SCORE_CHANGED, this.score)` - and `off()` on `SHUTDOWN`



Summary

- `time.addEvent` spawns; physics groups give members the same motion
- pool what you make **many of, often:** `classType` , `maxSize` , `get()` , `enableBody` / `disableBody`
- reset **everything** on reuse; `killAndHide` leaves the body on
- waves: staggered tweens, repeating timers, paths - counted down to zero
- fire rate and power-ups = stored times
- grow with data, phases, small classes, and a HUD that listens

Challenges

1. **Pointer control** - the basket follows the mouse or a finger (catcher)
2. **Combo scoring** - 10, 20, 30... for fruit in a row (catcher)
3. **Zig-zag enemies** - a third wave pattern (intermediate)
4. **Shield power-up** - takes the next hit (advanced)
5. **Smart bomb** - B clears the screen, three a game (advanced)
6. **Second player** - A / D / W, two scores (intermediate)

Next: **Chapter 15 - Platformer**